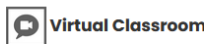
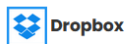




LAB 2

MATERIAL AT



10 file caricati per un totale di 510.7 MB
Ultimo aggiornamento 30/09/2024 09:23:12

Cerca...



- /
- Exercises
- IDE Setup
- Labs
- Slides

Nome file ▼	Dimensione	Data	Accesso	Visibilità
Exercises		27/09/2024 18:15:17		
IDE Setup		30/09/2024 09:13:40		
Labs		30/09/2024 09:17:32		
Slides		25/09/2024 12:26:59		

Extracting a character from a String - RECAP

- Each character inside a String has an index number:

0	1	2	3	4	5	6	7	8	9
c	h	a	r	s		h	e	r	e

index
character

- The first character has index zero (0)
 - The last character has index $\text{len}(\text{name}) - 1$
- The `[]` operator returns a character at a given index inside a String:

```
name = "Harry"  
first = name[0]  
last = name[4]
```

0	1	2	3	4
H	a	r	r	y

↑
name[0]

↑
name[4]

String slices - RECAP

- It is possible to extract a part of a string or *slice*
- Given a string:
 - `name = "never odd or even"`
- If we want the string slice from character 4 (index 3 → 'e') included, to 8 (index 7 → 'd') not included
- `stringSlice = name[3:7]`
- The string slice contains characters in indexes 3, 4, 5 and 6.
 - `"er o"`

String slices – 2 - RECAP

- Examples
- `name[ : 6] → "never odd or even"`
 - All the characters included from the first one (index 0) to the 6° one, (index 6 not included)
- `name[6 : ] → "never odd or even"`
 - Characters from the 7° to the last one included
- `name[ : ]`
 - All the characters included, it makes a copy of the string

String portions - RECAP

- A string portion can be extracted providing *start*, *stop* and *step* indexes
- `name[3:8:2] → "never_odd_or_even"`
 - Include all the characters from 3 to 8 using an increment of 2 of the index steps.
- `name[::2] → "never_odd_or_even"`
 - Include all the characters in even indexes
- `name[1::2] → "never_odd_or_even"`
 - Include all the characters in odd indexes

Exercise 02.1.1

- Write a program that stores two integers in two constants defined in the code , and then displays:
 - The sum
 - The difference
 - The product
 - The average value
 - The distance (i.e. the absolute value of the difference)
 - The maximum value (i.e. the greater of the two)
 - The minimum value (i.e. the smaller of the two)

Exercise 02.1.1 cont'd

- *max()* and *min()* functions defined in Python .
- They accept a sequence of comma-separated values as input and return the maximum and minimum values of the sequence respectively.
 - Ex: `max (10, 5, 3)` returns 10.

Exercise 02.1.1 cont'd

- **Pay attention to the assignments!**

- They should be "read" from right to left

< variable > = < expression >



- We can use a variable or calculate the result directly inside the ***print()*** function.

```
sum = a + b
```

```
print ( "The sum is:" , sum )
```

or

```
print ( "The sum is:" , a + b )
```

Exercise 02.1.5

- Electric force. According to Coulomb's law, the electric force (expressed in Newtons) between two charged particles with charge, respectively, Q_1 and Q_2 , separated by a distance r , is:

$$F = \frac{Q_1 Q_2}{4\pi\epsilon r^2}$$

- Where $\epsilon = 8.854 \times 10^{-12}$ Farad / meter.
- Write a program that calculates and displays on screen the relative force between a pair of charged particles, allowing the user to choose the values Q_1 , Q_2 (in Coulombs) and r (in meters). [P2.43]

Exercise 02.1.5 cont'd

- The goal of the exercise is to take familiarity with constants real and expressions
- It's enough apply the formula...

Exercise 02.2.1

- Write a program that stores a string in a variable and , from that variable , prints first 3 characters of the string , followed by 3 dots , again followed by the last 3 characters .
- For example , if the string is initialized to the value “Mississippi”, the program must display “Mis...ppi ”.

Exercise 02.2.1 cont'd

- Say that city is the string inserted , the initial characters can be easily extracted as:
 - `first = city[0]`, `second = city[1]` , etc.
- And the final characters ? In the case of “mississippi” (9 characters) the last character is `city[8]`, but in general strings **do not have all the same length**.
 - Our program must work for **any string!**
- **ALTERNATIVE: use slicing**

Exercise 02.2.1 cont'd

- After understanding how to extract the two substrings we have to chain them together adding “...” between them
- Remember that **math operators** assume different meanings depending on the type of the data to which they are applied .

Exercise 02.2.2

- The following pseudocode describes how to transform a string containing a ten-digit telephone number (such as “4155551212”) into a more readable, U.S.-style string, such as “(415) 555–1212”:
 1. Take the string consisting of the **first 3 characters** and add round brackets around it (this is the prefix, called “area code”);
 2. Concatenate the **prefix** with the string containing the **next 3 characters**, followed by a **hyphen**, and then the string consisting of the **last 4 characters**.
- Translate this pseudocode into a Python program that stores a 10-digit phone number in a string, then displays it in the format just described. [P2.33]

Exercise 02.2.2 cont'd

- It 's about interpreting the pseudocode

Take the string consisting of the first 3 characters and surround it with round brackets (this is the prefix, called “area code”):

- **First 3 characters = with slicing**
- **Add round brackets = concatenation**

Concatenate the prefix with the string containing the next 3 characters, a hyphen and the string consisting of the last 4 characters:

- **Remaining characters of the string = with slicing**
- **First 3 characters of the remaining string = with slicing**
- **Concatenate with hyphen and add to the previous one**